

Spring Boot 3 JPA & Backend Summary with Flow Diagrams

1. JPA & Entity Relationships

- JPA abstracts database operations using ORM. - Key annotations: @Entity, @Id, @GeneratedValue, @OneToMany, @ManyToOne, @JoinColumn. - Common strategies: FetchType.LAZY vs FetchType.EAGER. - Avoid N+1 problem using JOIN FETCH or @EntityGraph. - DTO Projections reduce data transfer and decouple persistence layer from API layer.

2. Spring Data JPA Specifications

- Allows dynamic, type-safe query construction. - Combine reusable filters: Specifications.where(nameContains(q)).and(emailDomain(domain)). - Useful for advanced filtering with pagination.

3. REST API Flow

1. Client sends HTTP request → Controller. 2. Controller → Service → Repository → Database. 3. Repository fetches Entities, maps to DTOs. 4. Response wrapped in HATEOAS (if enabled).

4. Interceptor Flow

- Interceptors act as middleware for REST APIs. - Can log requests, add headers, handle auth, measure execution time. - Order: PreHandle → Controller → PostHandle → AfterCompletion.

5. Kafka Event Flow

1. REST API receives request. 2. Service saves data → publishes event to Kafka topic. 3. Kafka consumer listens and processes event asynchronously. 4. Useful for decoupling microservices & async workflows.

6. Pagination & Sorting

- Pageable interface with PageRequest.of(page, size, Sort.by("field")). - Reduces memory usage and improves performance. - Supports cursor-based pagination for large datasets.

7. HATEOAS

- Hypermedia as the Engine of Application State. - REST responses enriched with navigation links. - Helps clients explore API dynamically.

8. Transactions

- @Transactional ensures atomic DB operations. - Propagation types: REQUIRED, REQUIRES_NEW, NESTED. - Rollback strategies for error handling.

Flow Diagrams

REST API Request Flow

Client → Controller → Service → Repository → Database Response → DTO Mapping → HATEOAS Wrapper → Client

Interceptor Flow

PreHandle → Controller → PostHandle → AfterCompletion

Kafka Event Flow

Producer (Service Layer) → Kafka Topic → Consumer (Async Processing)

Pagination & Sorting Lifecycle

Client Request (page, size, sort) → Repository Query (Pageable) → Page → Client